Which bits can we drop? Supplementary material

Detail that does not fit in the two-page report. CS6886, November 2026.

1 What this document adds

The submitted report has two pages. This document has the method detail, the full result tables and the engineering record. Read the report first. Use this document to check a number or to repeat the work.

The project asks one question. Which parts of a learned image codec can use a low precision? The answer is an assignment: one precision for each part. We measure the cost of each choice, solve the assignment as an integer program, and then test whether fine-tuning can recover the remaining cost.

2 Method

2.1 Measurement

We quantize one part at a time. All other parts and all activations stay in FP32. We then run the complete codec on each image at each quality setting. The plan covers seven formats, 19 parts, two datasets and eight settings. That gives 69,233 rows.

Two safeguards make a row valid. First, an FP32 gate must reproduce the reference bitstream and the reconstruction at all eight settings. Second, a hash check must confirm each weight restoration. No run failed this check.

2.2 The quality metric

We need one number that holds a quality change and a bitrate change together. We use a rate-equivalent penalty:

```
penalty = 100 * [ (1 + d_bpp/100) * 2^(-d_PSNR / slope) - 1 ]
```

The value is zero when the model is equal to FP32. The optimizer limits the worst setting. The worst setting is the maximum over the eight settings of the mean over the images. We also report Bjontegaard delta-rate as a second view, so that no claim depends on one metric.

2.3 Additivity

The optimizer adds single-part costs. That step is correct only if the costs add. We tested the condition. Divide the measured penalty of a uniform INT8 model by the sum of the 19 single-part penalties. The ratio is 0.96 to 1.05 on Kodak and 1.02 to 1.11 on CLIC.

One warning. Do not add the worst setting of each part. That sum gives 65.3 percent, but the true value at one setting is 26.7 percent. Add the costs for one setting, then take the maximum.

2.4 The allocation program

One weight file serves all eight settings. A different assignment for each setting is therefore not deployable. The model uses one binary variable for each pair of a part and a format. Each variable is shared by all settings. The model has one quality constraint for each setting.

The objective is the number of bytes. It is not latency. We have no measured latency table for each format on this GPU. An objective that used such a table would optimize data that we invented. We solve with HiGHS, and we also solve a greedy baseline for comparison.

2.5 Fine-tuning

We apply quantized convolutions with a straight-through gradient. We then minimize distortion and hold the bitrate at its pre-quantization value. A fixed rate penalty failed in both directions. A weak penalty lets the model move down the rate-distortion curve. A strong penalty destroys quality to obey the rate target. We use an augmented Lagrangian instead. The multiplier comes from the measured rate error on a fixed validation set.

We freeze the entropy parts. The bundle contains integer CDF tables. If the entropy weights move, those tables no longer describe the weights, and the encoder and the decoder disagree. The saved checkpoint must be better than its own start. If it is not, we keep the start weights. A bad run therefore cannot ship a worse model.

Training uses 100 DIV2K images that are disjoint from all test sets. We run 12,000 steps on 512-pixel crops. That takes about 100 minutes for one tier on one 4 GB laptop GPU. This work uses 2.2 percent of the optimizer steps and 0.2 percent of the sample budget of the pretraining. Thus it is a repair operation, not a new training.

3 Results

3.1 The frontier

Point	Size	Saving	Kodak	CLIC	Set14
1% budget champion	21.677 MB	-60.4%	0.672%	0.851%	0.935%
1.8% budget variant	18.664 MB	-65.9%	1.268%	1.595%	1.601%
INT8 tier + QAT	14.401 MB	-73.7%	1.848%	2.874%	3.363%
uniform INT8	13.904 MB	-74.6%	26.184%	56.701%	19.374%
safe, no fine-tuning	17.904 MB	-67.3%	2.633%	4.301%	2.810%
greedy mixed	24.565 MB	-55.1%	0.386%	0.358%	0.393%
uniform FP16	27.416 MB	-49.9%	0.038%	0.047%	0.056%

Table 1. Measured configurations. Values are worst-setting rate-equivalent penalty against a 54.688 MB FP32 baseline. Source: results/mixed/, results/final/.

The champion predicted 0.9715 percent and measured 0.94 percent. The prediction is conservative on each dataset. We did not use Set14 for fitting, so the result there is a transfer test. It holds.

3.2 The knee

Budget	Size	Predicted penalty
1.0%	21.677 MB	0.9715%
1.2%	21.396 MB	1.1374%
1.4%	21.258 MB	1.2851%
1.6%	21.151 MB	1.4437%

1.8%	18.664 MB	1.7162%
2.0%	18.389 MB	1.9864%

Table 2. A fine budget sweep. Between 1.6 and 1.8 percent the size falls 2.5 MB in one step. Source: results/night/ilp_fine_sweep.json.

The 1.8 percent assignment beats the 2 percent assignment that we first chose. It is 0.275 MB larger and better on all three datasets. We replaced the 2 percent point. This is why a frontier must be swept and not guessed.

3.3 Fine-tuning results

Tier	Size	Before	2,000 steps	12,000 steps	Damage removed
More INT8	14.401 MB	8.61%	8.61%	3.36%	61.0%
Moderate INT4	13.572 MB	119.18%	25.27%	20.53%	82.8%
More INT4	11.369 MB	1771.71%	84.22%	41.84%	97.6%
ILP 2% assignment	18.389 MB	1.936%	not run	1.936%	0.0%

Table 3. Fine-tuning outcome. The ILP row used 8,000 steps. Source: results/final/qat_attempt/summary.csv, results/night/qat_int8_12k/summary.csv.

QAT removes most of the damage on assignments that a person chose. It removes no damage on the assignment that the solver optimized. Therefore fine-tuning replaces a good allocation. It does not extend it. The assignment decides the size of the repair.

3.4 Group scaling for 4-bit

Part	INT8 penalty	cost/MB	int4g32 penalty	cost/MB
E3	0.427%	0.14	34.50%	31.4
E2	0.509%	0.17	28.85%	26.1
E4	0.710%	0.24	34.60%	31.2
E1	1.352%	0.46	172.27%	157.5
H3	38.152%	184	not viable	-
D5	8.057%	1712	not viable	-

Table 4. Cost per megabyte saved. Group-32 scaling cut INT4 weight error by 40 percent and damage by 5 to 22 times. Source: results/qat/WHY_THESE_COMPONENTS.md.

Per-output-channel INT4 measured 148 to 3767 percent penalty. Group scaling made it much better. It is still about 100 times the cost of the 8-bit step for each megabyte saved. So 4-bit is not worth shipping without fine-tuning.

3.5 Latency and energy

No configuration was faster than FP32. We used paired interleaved benchmarking with 50 pairs for each cell. The mixed model is 4.79 percent slower, with a 95 percent confidence interval of +1.27 to +9.80. Its energy change is -0.39 percent, and the interval includes zero.

The cause is arithmetic. A decode takes 88 ms. Entropy coding takes 80 ms of that time. The convolutional network takes 8 ms. Convolutions are 9 percent of a decode and 27 percent of a round trip. Quantization makes convolutions cheaper, so it cannot fix the bottleneck.

We make no speed claim for FP8, INT4 or FP4. These formats run as simulated quantization. Only FP16 and BF16 use real kernels on this GPU. They save energy, not time.

4 Engineering record

The pipeline did not work on the first attempt. Three failures did not show an error message. We record them because each one is a general trap.

- Checkpoint selector floored at zero. The selector accumulated max(0, penalty). A negative penalty means an improvement, so the selector could never choose an improved checkpoint. Each run before the fix saved its step-0 weights and discarded its own gain. This one bug cost most of one night.
- Two different bitrates. The training loss sees a surrogate rate of about 0.8. The measured stream bitrate is about 0.32 for the same model. A comparison of the two drove the Lagrange multiplier in one direction on a false error.
- An inference-mode trap. The codec compress helper runs under inference mode. If you call it between training steps, it caches inference tensors on the entropy model, and the next backward pass fails. Validation must call the model methods under no-grad instead.

We also made a wrong conclusion and corrected it. At 2,000 steps the INT8 tier gained nothing, and we concluded that QAT only helps severe damage. At 12,000 steps the same tier removed 61 percent. The first result was a step-budget effect, not a property of the model.

5 Limitations

- Checkpoint selection under-reports the fine-tuning gain. A probe with 6 crops rated the best INT8 model as equal to its start (-0.08 percent). The full evaluation shows 8.61 to 3.36 percent. The saved checkpoints are probably not the best available.
- Weights only. Activations stay in FP32, so we do not test W8A8. W8A8 is the configuration that would use an integer kernel.
- The 4-bit formats are simulated. Their quality numbers are real. Their speed is not modelled.
- The entropy coder uses one lane. Multi-lane coding is the largest unused latency lever, and it changes the bitstream contract. It is a design decision, not a tuning step.
- The fine-tuning results use one seed and 100 images. The direction holds for three tiers and two step counts. The exact percentages are single-run values.

6 How to reproduce

```
.\venv\Scripts\python.exe quantization-pluse-mse\scripts\eval_assignment.py --model
checkpoints/pulse-mse --configs quantization-pluse-mse\results\ilp\assignment_ilp.json --dataset
kodak --output results\check
```

The command gives 21.677 MB and 0.672 percent on Kodak, inside the FP32 gate. RUNBOOK.md has every other command, the expected runtime, and a troubleshooting list.

The checkpoint is not tracked in the repository because it is 84 MB. Download the released bundle from https://huggingface.co/zhaoyangjia/PULSE. Place three files in checkpoints/pulse-mse/. The file entropy_control_int.pt is mandatory. The loader refuses a bundle without it, because a run without it would produce a different bitstream than the published model.

Code: https://github.com/harshith0518/CS6886-PULSE-Quantization